

Risposta alle domande G2 di Sistemi Operativi

Nicolò Giuliani

Indice

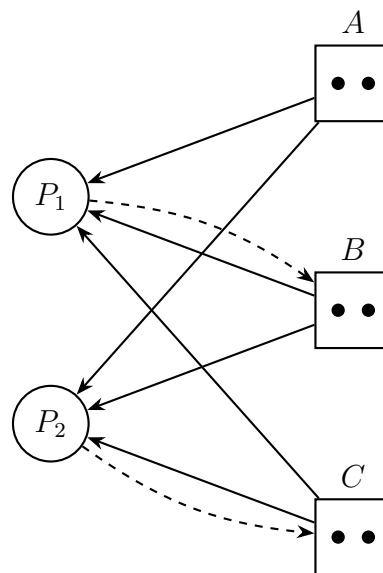
06 febbraio 2026	2
09 gennaio 2026	3
05 settembre 2025	4
21 luglio 2025	6
23 giugno 2025	7
28 maggio 2025	8
11 febbraio 2025	9
15 gennaio 2025	10
10 settembre 2024	11
22 luglio 2024	12
25 giugno 2024	13
29 maggio 2024	14
13 febbraio 2024	15
17 gennaio 2024	16
11 settembre 2023	17
	18

- a) In una tabella FAT possono esistere elementi di uguale valore? (non considerando l'elemento nullo che contraddistingue la fine del file) Se sì: in quali casi? Se no: quali effetti avrebbe?

R: No, in FAT non possiamo avere elementi di uguale valore, infatti oltre a *EOF*, che serve per segnare il termine di una catena di puntatori. Se avessimo elementi di ugual valore vorrebbe dire che due puntatori indicano lo stesso blocco. Questo significherebbe che un blocco è stato sovrascritto da un altro file; ovvero uno o più file sono corrotti.

- b) Trovare, se possibile, un grafo di Holt che abbia 5 nodi, tutte le risorse abbiano molteplicità due (due unità per ogni classe di risorse) e tutti i processi siano in deadlock

R: Grafo di Holt con 2 processi + 3 classi di risorse (molteplicità 2) in deadlock.



- c) Perché i metodi di sincronizzazione tipo test&set (detti anche spinlock) assumono grande rilevanza nella scrittura di sistemi operativi multiprocessore?

R: Nei sistemi multiprocessore dobbiamo implementare il kernel in multiprocesso concorrente. Per non avere conflitti tra thread del kernel dobbiamo avere accesso atomico alle strutture dati del Kernel. Per far questo un semplice semaforo non basta dato che blocca a livello solo della singola CPU. Mentre con gli *spinlock*, ovvero istruzioni dell'IS della CPU, abbiamo accesso atomico su tutti i processori logici contemporaneamente.

- d) Perché è difficile revocare una autorizzazione fornita tramite capability?

R: Le capability non hanno una "sede centrale" dove possiamo gestirle tutte come per ACL, infatti ogni processo/utente (e anche i figli che la ereditano) ha la sua capability. Quindi in un sistema multiprocesso/multiutente diventa impossibile (o quasi) fare una revoca perché bisognerebbe scansionare l'intero disco (e sapere anche i figli).

a) Quali problemi possono accadere se si utilizza un file system ext2 dove un file ha un numero di riferimenti errato?

R: *Il numero di riferimenti in ext2 indica il numero di hard link del file, un numero errato comporta che abbiamo realmente piu' o meno hardlink.*

1. *Caso piu' hardlink: il fs segna che abbiamo piu' hardlink, quindi se andiamo a eliminare l'ultimo vero hardlink il fs non eliminerà internamente l'inode e non lascerà i blocchi come liberi, portando a un inutile memoria occupata per sempre.*

2. *Caso meno hardlink: il fs segna che abbiamo meno hardlink, quindi quando andiamo a eliminare un hardlink (anche se realmente ne abbiamo altri) il fs eliminerà l'inode e metterà i blocchi come liberi, portando a una corruzione del file.*

Per fixare entrambi i casi fsck (File System Consistency Check) scannerizza tutti i file e per ogni file conta il numero di hardlink reali e poi corregge se serve nel inode.

b) Siano dati due array di dischi identici, il primo viene gestito in modalità RAID1, il secondo in modalità RAID5. In quale array l'operazione di lettura è statisticamente più veloce?

R: *Il RAID1 ha come massimo di velocità in lettura la velocità di lettura minore di tutti i dischi. Mentre RAID5 avendo oltre che ridondanza anche uno striping abbiamo velocità in lettura maggiori rispetto anche al singolo disco.*

c) La seguente descrizione del funzionamento dell'algoritmo del banchiere è errata: "Quando un processo richiede risorse, l'algoritmo del banchiere controlla se lo stato del sistema rimarrebbe safe assegnando le risorse richieste: se sì le risorse vengono assegnate, se no l'operazione viene sospesa. Quando un processo restituisce risorse si controlla se il nuovo stato consente di soddisfare una o più operazioni sospese." Perché è errata? Come si può correggere la descrizione?

R: *Per la frase "... Quando un processo restituisce risorse si controlla se il nuovo stato consente di soddisfare una o più operazioni sospese." Bisognerebbe dire "... Quando un processo restituisce risorse si riesegue l'algoritmo del banchiere per vedere se il nuovo stato è safe o unsafe."*

d) A cosa serve e come funziona il meccanismo di Aging negli scheduler a priorità dinamica?

R: *Negli scheduler con priorità può accadere che i processi a bassa priorità non vengano mai eseguiti perché i processi ad alta priorità occupano sempre lo scheduler. Quindi per non far accadere questo con il passare del tempo i processi aumentano gradualmente la propria priorità fino a raggiungere in caso quella massima. Così tutti i processi verranno eseguiti.*

- a) Quando è sconsigliato usare un meccanismo di protezione di accesso di tipo Access Control List?

R: *Se abbiamo bisogno di semplici permessi `r/w/x` per `owner/group/others` i permessi Unix bastano e le ACL aggiungono solo complessità e overhead. Per i servizi come docker o i file-sharing le ACL non funzionano. E rispetto le capability se vogliamo lavorare su processi e consentire ai processi singoli permessi speciali con capability possiamo mentre con ACL no.*

- b) Qual è la differenza fra una funzione di una libreria ed una system call? In altre parole, perché sono necessarie le system call?

R: *La libreria generalmente sono fatte in:*

$$User\ space \rightarrow Syscall\ (kernel\ space) \rightarrow User\ space$$

*e quindi le librerie sono dei **wrapper** che ampliano le syscall.*

Abbiamo però sempre bisogno delle syscall infatti queste ci permettono di andare in kernel space per poter accedere alle strutture protette. Questo complessivamente è un meccanismo di protezione e di agevolamento al programmatore che seguendo la filosofia della OOP non deve preoccuparsi delle implementazioni interne.

- c) Il problema del deadlock può essere risolto con tecniche di checkpointing & rollback?

R: *Sì, però questo consiste in snapshot periodici del sistema, rilevazione runtime del deadlock e ripristino del sistema (recovery). Questo porta come vantaggio il non dover conoscere a priori le risorse di ogni singolo processo rispetto al algoritmo del banchiere (preemption). Però porta un overhead di memoria per gli snapshot. Questa tecnica può essere ulteriormente ottimizzata nei linguaggi reversibili con modelli:*

```
try(condition){  
    //do something that induce deadlock  
} else{  
    //rollback  
}
```

Questo modello non porta overhead di memoria dal rollback dato che non abbiamo snapshot, ma porta a un costo di esecuzione maggiore perché rilevato un deadlock dobbiamo invertire tutte le operazioni per tornare alla situazione iniziale. Oltre al fatto che tutte le operazioni debbano essere invertibili per natura (a meno che di compilatori che traducono under-the-hood il linguaggio in uno reversibile per poi eseguirlo su interprete).

$$\begin{aligned} f(x) &\rightarrow f'(x) \\ f'(x)^{-1} &\rightarrow f(x) \end{aligned}$$

ovvero

$$f(x) \iff f'(x)$$

d) Dati N dischi aventi le stesse caratteristiche, il tempo medio di accesso è minore se creiamo con essi un cluster RAID0 o uno RAID5? Discutere la perdita di prestazione per la lettura e la scrittura.

R: Con RAID0 abbiamo N accessi, perché ogni file è diviso in striping su tutti i dischi. Mentre in RAID5 abbiamo 2 accessi ovvero disco dove scriviamo il blocco e il disco di parità.

Lettura:

- RAID0: N accessi paralleli
- RAID5: 1 accesso

Scrittura:

- RAID0: N accessi paralleli
- RAID5: 4 accessi sequenziali

$$P_{new} = P_{old} \oplus D_{old} \oplus D_{new}$$

Con P e D rispettivamente per parità e dato.

A livello di tempo medio il RAID0 vince.

21 luglio 2025

a) Perché durante la compattazione di memoria occorre sospendere l'esecuzione dei processi coinvolti?

R: *Perché i processi portano a frammentazione esterna e quindi si verifica un caso del genere:*

Prima: [P1][—][P2][—][P3]

Dopo: [P1][P2][P3][—]

Quindi la compattazione sposta i blocchi di memoria dei processi in una sequenza contigua per ottimizzare lo spazio totale. Però facendo questo gli indirizzi cambiano. Quindi il processo bisogna fermarlo, compattare, calcolare i nuovi indirizzi, applicarlo al processo.

b) Perché nell'i-node non è presente il nome del file?

R: *Nell'inode non memorizziamo il nome del file ma il contatore degli hardlink. Infatti se memorizzassimo il nome del file non avremmo la opzione degli hardlink (come per FAT), mentre attraverso il contatore degli hardlink questo è possibile. Il nome invece è memorizzato nella directory e anche il rispettivo inode.*

c) Perché il microkernel è meno efficiente del kernel monolitico? Perché è più flessibile e sicuro?

R: *Il mikrokernel avendo solo il minimo indispensabile in kernelspace e quasi tutti i driver, programmi, ecc in userspace per motivi di flessibilità, passa molto tempo a fare lo swap da usermode a kernelmode (perché richiamato molte volte) e quindi porta molto overhead. Però è decisamente più flessibile perché se vogliamo modificare un driver non dobbiamo ricompilare tutto il kernel ma solo il singolo programma. È anche più sicuro perché solo pochi componenti del SO possono accedere alle strutture protette del kernel, rendendo quindi la modifica/lettura molto più difficile.*

d) Nella memorizzazione da parte dei processi utente delle capability per il controllo di accesso può essere usato un algoritmo di crittografia a singola chiave (chiave privata) oppure occorre un algoritmo a doppia chiave (chiave pubblica)? Perché?

R: *Esclusivamente a chiave privata, perché il kernel firma la capability con la chiave privata (nascosta) e gli utenti/processi controllano la creazione sicura attraverso la loro pubblica. Questo serve perché se i processi/utenti conoscessero la chiave unica allora potrebbero creare loro una capability e firmarla con essa.*

23 giugno 2025

a) L'algoritmo LOOK (detto anche dell'ascensore) è più efficiente di C-LOOK (che ha lo stesso principio di funzionamento del metodo LOOK, ma la scansione del disco avviene in una sola direzione). In quali casi si preferisce C-LOOK a LOOK?

R: *LOOK ha un throughput maggiore ma il tempo tra le richieste agli estremi è molto alto, mentre C-LOOK va a risolvere il problema andando a minimizzare la varianza (differenza dei tempi medi).*

b) Spiegare quali attacchi sono possibili se non viene usato il meccanismo del "sale" (salt) nella memorizzazione delle password criptate.

R: *Il salt serve per proteggerci dagli attacchi dizionario. Infatti la password viene confrontata con $H(\text{password} + \text{salt})$ con salt come numero univoco per utente. questo numero è di 256bit\32byte.*

c) L'algoritmo del Banchiere evita il verificarsi di situazioni di deadlock ritardando le richieste che porterebbero il sistema in uno stato unsafe. Basta questa regola a evitare anche situazioni di starvation?

R: *No, infatti la richiesta potrebbe essere rimandata all'infinito. Bisognerebbe implementare anche una tecnica simile al aging per impedire starvation.*²

d) Perché è necessario usare spinlock in sistemi multiprocessore per implementare kernel di tipo simmetrico (SMP)?

R: Vedi risposta a pagina 2, esame 06 febbraio 2026.

a) Perché è difficile revocare un diritto di accesso se espresso come capability rispetto a revocare lo stesso diritto in una access control list?

R: *Perche' in capability non abbiamo una lista centrale per file/dir. come per ACL con tutti i permessi li, ma tante capability per diversi processi/utenti (e anche per i figli). Quindi la revoca diventa impossibile. Mentre per ACL ci basta modificare la tabella riferito a quel file.*

b) Si può implementare la memoria virtuale con paginazione a richiesta in un sistema che ha una Memory Management Unit priva di TLB (Translation Lookaside Buffer)?

R: *Si, pero' senza la TLB come buffer bisogna ogni volta leggere dalla tabella in RAM. Questo porta' un grande overhead.*

c) Perché non si può implementare la funzionalità di link fisico di file in un file system di tipo FAT?

R: *In FAT non esiste un reference counter associato ai blocchi: l'unica informazione presente è la catena nella tabella FAT. Se due directory entry puntassero forzatamente alla stessa catena, al momento della cancellazione di una delle due non ci sarebbe modo di sapere se esistono altri riferimenti, quindi i blocchi verrebbero liberati anche se l'altra entry li sta ancora usando. Per supportare gli hard link servirebbe un contatore di riferimenti, che FAT non prevede.*

d) L'algoritmo del Banchiere evita che si verifichino situazioni di deadlock. Sembra una funzione desiderabile eppure viene raramente usato nei sistemi reali, perché?

R: *Perche' nei sistemi reali e' quasi impossibile sapere a priori il numero di risorse che un algortimo ha bisogno, e quindi le condizioni per eseguire l'algortimo diventano impossibili. Poi c'e' anche da tenere in mente il costo computazionale alto perche' per n processi e r valute il costo e' di $O(n^2 \cdot r)$.*

11 febbraio 2025

a) La tecnica dell'aging serve per evitare casi di starvation. Come funziona?

R: *Si, la tecnica del aging serve per gli scheduler a priorita', nei quali se non avessimo aging alcuni processi potrebbero non essere mai eseguiti perche' i processi a priorita' maggiore occupano tutto il tempo la CPU. Quindi con il passare del tempo l'aging aumenta la priorita' di un processo che non e' stato eseguito.*

b) Può esistere un controller di device capace di usare DMA ma che non genera interrupt? Se NO: perché non esiste? Se SÌ: come deve essere costruito il relativo device driver?

R: *Puo' esistere ma allora in quel caso dovremmo avere la CPU che fa polling costantemente per vedere quando il trasferimento e' completato. Questo andrebbe ad annullare tutti i benefici del DMA. Quindi gli interrupt ci servono proprio per mettere in pausa la CPU e farla continuare con altri processi che non tocchino gli indirizzi che stiamo lavorando con il DMA.*

c) Quanti dischi sono coinvolti da una operazione di scrittura di un blocco in un array di dischi RAID 5? Perché?

R: *In scrittura in RAID5 abbiamo 4 operazioni sequenziali:*

- 1. Lettura blocco dati vecchio*
- 2. Lettura blocco parita' vecchio*
- 3. Scrittura dati nuovo*
- 4. Scrittura parita' nuova*

$$P_{new} = P_{old} \oplus D_{old} \oplus D_{new}$$

con P e D rispettivamente per parita' e dati.

d) Quale problema risolve la compattazione di memoria? Quali sono i difetti di questo metodo?

R: *La compattazione di memoria serve per ridurre la frammentazione esterna. Questa puo' essere fatta sia su dischi fissi, ma generalmente su RAM per i processi su CPU senza MMU. Se avessimo una MMU a priori la compatazione non servirebbe perche' ci basterebbe modificare la tabella degli indirizzi virtuali a indirizzi fisici. Il difetto principale e' che dobbiamo fermare i processi, spostare i dati, ricalcolare i nuovi indirizzi e copiarli nella PCB del processo e poi riprendere l'esecuzione.*

15 gennaio 2025

- a) Un i-node di un file system tipo ext2 per un errore viene riportato un numero di link maggiore del reale. Cosa può succedere se si continua ad usare il file system? (perché?)
E se il numero di link errato fosse al contrario inferiore al reale cosa potrebbe succedere? (perché?)

R: Vedi risposta a pagina 3, esame 09 gennaio 2026.

- b) Perché è necessario usare spinlock in sistemi multiprocessore per implementare kernel di tipo simmetrico (SMP)?

R: Vedi risposta a pagina 2, esame 06 febbraio 2026.

- c) Perché nei sistemi reali l'algoritmo di rimpiazzamento second chance (orologio) viene preferito a LRU sebbene il primo non sia a stack e il secondo sì?

R: *Per implementare LRU dovremmo avere una MMU che memorizzi l'ordine di tutte le pagine. Per farlo abbiamo 2 opzioni:*

1. *Tabella $n \times n$, costo di lettura della pagina $O(1)$*
2. *Tabella $n \times 1$, costo di lettura della pagina $O(n)$*

In entrambi i casi il problema maggiore è il costo della memoria buffer della MMU, che deve essere una cache ad alte prestazioni con molta memoria (soprattutto per il caso n.1). E questo rende il sistema troppo costoso.

*Generalmente rispetto a LRU si preferisce il **second chance**, una tecnica che approssima abbastanza bene LRU però con*

- *n bit, costo di lettura $\Omega(1)$ e $O(n)$.*

Quindi il costo per implementare quest'ultima tecnica è decisamente minore e anche rispetto al n.2 di LRU ha tempi d'accesso minori ($\Theta(\frac{n}{2})$) e' migliore rispetto al $\Theta(n)$ del n.2).

- d) Perché revocare un'autorizzazione espressa come capability è più difficile che revocare lo stesso diritto quando espresso come access control list?

R: Vedi risposta a pagina 8, esame 28 maggio 2025.

- a) In RAID61 (RAID1 di RAID6) qual è il livello di fault tolerance (quanti dischi possono guastarsi)? E in RAID16 (RAID6 di RAID1)?

R: In **RAID61** (RAID1 di RAID6) si mantengono due copie identiche di un array RAID6. Poiché RAID6 tollera fino a 2 dischi guasti per array, nel caso peggiore garantito si possono perdere **almeno 2 dischi qualsiasi**. Nel caso limite, se un intero array RAID6 cade, l'altro lo copre interamente, quindi si possono perdere tutti i dischi di un array più 2 dell'altro.

In **RAID16** (RAID6 di RAID1) i "dischi" del RAID6 sono coppie RAID1. RAID6 tollera la perdita di 2 array interi, e ogni coppia RAID1 tollera 1 disco guasto internamente. Quindi si possono perdere entrambi i dischi di al massimo 2 coppie (assorbiti dal RAID6), più un disco da ciascuna delle coppie rimanenti (assorbiti dal RAID1). La fault tolerance scala con il numero di coppie, rendendo RAID16 generalmente più robusto di RAID61.

- b) Nella configurazione di un sistema l'amministratore decide di creare una partizione di un disco rotazionale con file system FAT da montare come /tmp. È una buona o cattiva idea? Perché?

R: E' una cattiva idea perche' in FAT per leggere il blocco n dobbiamo prima leggere n blocchi e i file sotto /tmp chiedono molte letture veloci di blocchi in qualsiasi posizione. Oltretutto per gli HDD peggiorerebbero molto le performance.

- c) Quali sono i benefici della gestione degli interrupt nidificati e quali sono le difficoltà da affrontare per sviluppare un sistema operativo che supporti gli interrupt nidificati?

R: Il beneficio maggiore è la maggiore responsività del sistema, infatti per gli interrupt di I/O questi possono avere priorità maggiore rispetto ad altri in corso e questo porta a un sistema più responsivo per l'utente. Per far questo ci servirebbe uno stack per poter segnare a quale interrupt tornare, ripristinare il sistema a quando era stato bloccato e non avere cicli infiniti internamente.

- d) Perché aumentando la dimensione dell'area di memoria secondaria usata dalla memoria virtuale si corre il rischio di thrashing?

R: Perché la CPU accetterebbe l'esecuzione di più processi e quindi più memoria utilizzata e quindi dovremo passare sempre più tempo a passare dati da RAM a memoria secondaria, e avverrebbe thrashing peggiorando le performance.

22 luglio 2024

a) La ricostruzione della coerenza mediante tecnica di journaling risolve solo alcuni tipi di incoerenze; quali non vengono gestite?

R: *Il journaling risolve le incrongruenze a livello di filesystem quindi come due file occupano lo stesso blocco (quindi uno o entrambi i file sono corrotti), il numero di hardlink (se supportati) e' errato, blocchi che vengono segnati come occupati ma sono liberi e viceversa. In casi come FAT blocchi che creano dei loop circolari e non sono segnati fra i liberi. Le incoerenze che non vengono gestite sono quelle a livello del SO ovvero per esempio un programma che nella scrittura errata via disco scrive i dati sbagliati, un software malevolo che cripta i dati, il sistema operativo si corrompe con un aggiornamento, se si corrompe in boot sector il fs non puo' ripristinarlo dato che e' una partizione esterna rispetto il fs; e altri ancora*

b) A cosa serve partizionare un disco? Fornire esempi pratici di uso di dischi partizionati.

R: *Il partizionamento del disco serve per due motivi principali: il primo e' una ottimizzazione nell'ultizzo del disco a livello del SO, ovvero dividere il disco il piu' partizioni permette di utilizzarlo al meglio. Infatti possiamo installare 2 SO diversi, partizionarlo per avere memoria SWAP, partizionarlo per /home o /tmp e in generale per poter dividere i dati in maniera piu' "comoda per un umano". Mentre un secondo motivo (ad oggi poco rilevante), che partizionando per esempio i blocchi piu' esterni di un disco (a parità di velocità angolare, le tracce esterne hanno maggiore velocità lineare) abbiamo un leggero miglioramento delle prestazioni.*

c) Il tempo rimanente calcolato dall'algoritmo SRTF (shortest remaining time first) può diventare negativo? (sì o no, perché?)

R: *No, perche' SRTF calcola*

$$\min_x \text{Tempo rimanente di } x = \{ \text{Tempo stimato di } x - \text{Tempo passato di } x \}$$

con $x \in$ Processi in coda dello scheduler.

Quindi dato che il tempo stimato e' massimo $\geq$ del tempo passato (il tempo stimato si aggiorna il realtime), e quindi il tempo rimanente e' sempre ≥ 0 .

d) I sorgenti del sistema operativo Linux possono essere scaricati e il kernel può essere ricompilato. In quali casi pensate sia necessario compilare un kernel specifico?

R: *Dipende fortemente dal tipo di kernel. Il kernel linux e' un caso particolare perche' non e' il classico kernel monolitico che va sempre ricompilato ad ogni aggiornamento del SO o dei driver, infatti Linux attraverso i moduli va ricompilato solo quando si aggiorna (o in generale si modifica) la logica interna (come scheduler, TLB, gestione dei raw device, ecc). Diversamente ancora e' per i microkernel che non vanno quasi mai ricompilati (tranne per le modifiche a livello del SO, come per linux).*

25 giugno 2024

- a) L'algoritmo di rimpiazzamento *LeastUsed* sceglie come pagina vittima quella che ha avuto il minor numero di riferimenti da quando è in memoria. Quali funzionalità dovrebbe avere la MMU per poter implementare questo algoritmo di rimpiazzamento? Possono esistere MMU che forniscono le funzionalità richieste?

R: *L'algoritmo **LeastUsed** deve memorizzare in una tabella $n \times 1$ il numero di accessi per ogni pagina, quindi abbiamo anche bisogno di una logica di incremento oltre alla memoria e confronto oltre al buffer di memoria. Per implementarlo nella realtà è quasi infattibile perché anche se teoricamente va bene, i limiti fisici dei componenti, sia per il costo alto della memoria, sia per quello di sommatorio e confronto estremamente veloci, portano a dei costi generali troppo alti per un sistema.*

- b) Perché è bene che l'area di memoria dove viene memorizzato lo stack di valutazione non sia eseguibile?

R: *Perché se fosse eseguibile attraverso un buffer overflow potremmo cambiare il **return address** del RDA e quindi poi puntare a un blocco di codice malevolo.*

- c) RAID0 non tollera guasti. Quali sono i casi di utilizzo di RAID0?

R: *RAID0 è pensato per le performance senza ridondanza, mentre altri sistemi RAID come il 5, 6, 10 ecc (anche le versioni in ZFS) sono una via di mezzo tra performance e ridondanza. I casi possono essere trasferimento multimediale, server cache, gestione IO ad alte velocità, ecc.*

- d) Perché è necessario usare spinlock in sistemi multiprocessore per implementare kernel di tipo simmetrico (SMP)?

R: *Vedi risposta a pagina 2, esame 06 febbraio 2026.*

- a) Un controller DMA per i dischi migliora le prestazioni di un servizio di memoria virtuale con paginazione a richiesta?

R: *Un controller DMA può migliorare le performance della paginazione della memoria virtuale. Infatti senza DMA abbiamo:*

$$\text{RAM} \Rightarrow \underbrace{\text{CPU}}_{\text{collo di bottiglia}} \Rightarrow \text{DISK}$$

Mentre con DMA abbiamo:

$$\text{RAM} \Rightarrow \text{DISK}$$

e viceversa (in entrambi i casi).

Quindi DMA e i suoi interrupt serve per poter non far occupare la CPU cicli dove aspetta la fine del trasferimento, infatti mentre un trasferimento è attivo la CPU può fare altri processi nei quali non sono richiesti quegli indirizzi virtuali.

- b) Il meccanismo del sale (salt) serve per evitare gli attacchi di tipo “dizionario”. Perché un “sale” di un solo byte può non essere sufficiente?

R: *Perché con un solo byte possiamo scrivere $2^8 = 256$ numeri. E quindi l'attaccante potrebbe provare tutti gli Hash per i 256 numeri. Quindi aumentando anche solo a 16byte abbiamo $2^{8 \cdot 16}$ numeri che è un numero che computazionalmente è impossibile da trovare con il bruteforce a dizionario.*

- c) Per migliorare la gestione dei processi interattivi un progettista decide di implementare uno scheduler round robin dimezzando la durata del prossimo time slice a ogni processo ogni volta che lo consuma completamente senza fare I/O. Quali problemi potrebbero emergere?

R: *Per ogni processo senza IO facciamo che $\text{timeslice} /= 2$. Questo porta che il timeslice converga a 0. E porterebbe a un sistema dove non si eseguono più processi ma solo skip di essi, rendendolo inutilizzabile.*

- d) Che differenza c'è fra deadlock prevention e deadlock avoidance? Il metodo dell'allocazione gerarchica è un metodo di prevention o avoidance? Perché?

R: *Il deadlock prevention è una tecnica che previene anche solo una unica condizione del deadlock. Mentre l'avoidance previene tutte e 4 le condizioni necessarie (mutua esclusione, hold and wait di risorse, attesa circolare, no preemption). Il metodo della allocazione geradica previene l'attesa circolare.*

13 febbraio 2024

a) Perché è necessario l'interval timer per implementare lo scheduler round robin?

R: *Perché un processo in RR senza IT. non terminerebbe mai. Il context switch tra processi non avverrebbe e quindi nessun processo termina.*

b) Per accedere in modo diretto (seek) a un file di grandi dimensioni è più efficiente usare file system con allocazione concatenata o indicizzata? Perché?

R: *Per un file di grandi dimensioni l'allocazione indicizzata è superiore infatti in ext2 per accedere ai blocchi >65k abbiamo solo 4 letture, mentre in FAT (concatenata) abbiamo 65k letture.*

c) Per prevenire i deadlock si usa l'algoritmo del banchiere. Cosa fa il sistema operativo se una richiesta porta ad uno stato unsafe? Perché?

R: *Se il SO rileva un potenziale deadlock se rileva che lo stato è unsafe, semplicemente rifiuta la richiesta del processo e lo mette in coda insieme agli altri processi in coda. Quando ad una nuova richiesta/rilascio di risorse l'algoritmo verrà ri-eseguito il SO controllerà se esiste almeno una combinazione t.c. il sistema abbiamo abbastanza risorse da poter eseguire tutti i processi in quel ordine dato, così lo stato rimarrà safe.*

d) Dato lo schema di numerazione del sistema di archiviazione RAID, confrontare RAID 5 con RAID 50. Dato un array di dischi, quale metodo consente la maggiore capacità utilizzabile? Quanti dischi guasti sono tollerati? Quali vantaggi o svantaggi hanno i due metodi?

R: *Il RAID5 consente un solo disco di tolleranza, mentre il RAID50 due dischi nel caso ottimo (1 disco per ogni RAID5). Un RAID5 ci richiede solo 1 disco di parità mentre il RAID50 2. Il RAID5 ha meno overhead ma per tanti dischi le performance possono degradare, mentre il RAID50 ha senso quando abbiamo una pool di dischi grande e quindi dividendola in 2 RAID5 e poi lo striping otteniamo delle ottime prestazioni.*

17 gennaio 2024

a) Quali sono gli algoritmi di scheduling più adatti a sistemi interattivi, quelli preemptive o quelli non-preemptive? Perché?

R: *Per i sistemi operativo interattivi gli scheduler preemptive permettono di non avere delle attese troppo lunghe per le operazioni di IO. Quindi risultano decisamente più reattive all'utente. Questo perché magari un processo con tanto calcolo occuperebbe la CPU per molto tempo, ma con uno scheduler preemptive questo problema è solo parziale perché dopo un tot. di ms. cambiamo processo.*

b) Le unità di memoria secondaria allo stato solido (SSD) sono più veloci dei dischi. L'uso di SSD per implementare la memoria virtuale consentirebbe di avere minori problemi di thrashing (a parità di ampiezza della RAM e di memoria secondaria dedicata)? Perché? È comunque sconsigliato di usare SSD per gestire la memoria virtuale, perché?

R: *E' fortemente sconsigliato perché gli SSD hanno un target di scrittura/lettura sulle celle di memoria. Quindi anche se avremo prestazioni migliori, le operazioni di SWAP scriverebbero e ri-scriverebbero in continuazioni quelle celle. Portando a un usura veloce della memoria, fino a che dopo poco andrebbe ricambiata.*

c) Mostrare almeno tre tipi di incongruenze che possono essere rilevate durante il controllo di coerenza di un file system di tipo ext3.

R:

1. *Numero di Hardlink errati, bisogna per fixare ricontare manualmente in tutto il disco quanti hardlink reali abbiamo*
2. *Due inode che puntano agli stessi blocchi utilizzati, quindi uno o entrambi i file saranno corrotti. L'utente probabilmente dovrà fixare manualmente scegliendo lui*
3. *Blocchi segnati come occupati mentre solo liberi e viceversa, scansione dei blocchi realmente occupati e segnarsi realmente quelli liberi*

d) Dato lo schema di numerazione del sistema di archiviazione RAID, come funziona RAID 60? Qual è il numero minimo di dischi necessario? Quale è la capacità utilizzabile? Se tutti i dischi sono regolarmente funzionanti, a quanti dischi occorre accedere per effettuare una operazione di scrittura di un blocco dati?

R: *Il RAID0 di RAID6 comprende 4 dischi di parità, minimo 4*2 dischi. La capacità utilizzabile è $n - 4$ dischi. Se vogliamo scrivere un blocco dobbiamo:*

- *6 operazioni sequenziali per RAID6 (3 dischi)*
- *2 operazioni parallelo RAID0 (2 dischi)*

Il RAID0 scrive un blocco su un disco e l'altro blocco sull'altro disco. Quindi se dobbiamo scrivere un singolo blocco in RAID60 allora scriveremo un solo blocco nel primo RAID6, ovvero 3 dischi.

- a) Perché è necessario l'algoritmo della media esponenziale nel calcolo dello scheduler shortest job first? La media esponenziale dipende da un parametro (normalmente indicato con α , di valore fra zero e uno). A cosa serve questo parametro?

R: *SJF richiede di conoscere il burst CPU futuro di un processo, ma questo non è noto a priori. Si stima quindi basandosi sui burst passati. La media esponenziale permette di fare questa stima senza dover memorizzare tutta la storia dei burst — basta il valore precedente stimato e l'ultimo burst reale.*

$$\tau_{n+1} = \alpha t_n + \bar{\alpha} \tau_n \quad \text{con } \alpha \in [0, 1]$$

Quindi α serve per decidere quando "peso" dare al tempo reale rispetto al burst precedentemente calcolato.

- b) Dato un array di 4 dischi di uguale dimensioni confrontare l'uso di raid10 e di raid6. Qual è la capacità utilizzabile nei due casi? Qual è il livello di fault tolerance? Quali sono le prestazioni?

R: *In RAID6 abbiamo 2 dischi di ridondanza, quindi $N - 2$ dischi di spazio. Mentre per RAID10 abbiamo che ogni RAID1 ha 1 disco di ridondanza ma abbiamo 2 RAID1; quindi anche qua 2 dischi di ridondanza totale. Il RAID6 ha una tollerance migliore, perche' se anche i due RAID hanno 2 dischi di ridondanza, in RAID6 possiamo perdere 2 dischi qualunque. Mentre in RAID10 unicamente 1 disco per RAID1. Quindi nel caso ottimo 2 dischi, in quello limite 1 disco singolo. Le prestazioni in RAID10 sono buone, infatti il costo di scrittura e' basso, 2 operazioni per RAID1 quindi 4 operazioni (di cui 2 alla volta parallelizzate). Il RAID6 ha un costo di calcolo superiore, infatti 6 operazioni sequenziali.*

- c) Perché i metodi di sincronizzazione tipo test&set (detti anche spinlock) assumono grande rilevanza nella scrittura di sistemi operativi multiprocessore?

R: *Vedi risposta a pagina 2, esame 06 febbraio 2026.*

- d) Il file system ext2 è più efficiente nell'accesso diretto a file di piccole dimensioni rispetto a quelli di grandi dimensioni. Perché?

R: *Perche' in ext2 abbiamo 15 puntatori:*

- 0-11 puntatori ai primi 12 blocchi
- 12-267 puntatori con 1 tabella prima del blocco
- 268-65803 puntatori con 2 tabelle prima del blocco
- >65804 puntatori con 3 tabelle prima del blocco

Quindi per un file che sta nei primi blocchi rispetto a un file di grandi dimensione con piu' blocchi l'accesso al file comporta meno passaggi per puntatori.

a)

R:

b)

R:

c)

R:

d)

R: